

BUILDING THE CENTER OF EXCELLENCE FOR POWER PLATFORM

AFTER AGILE

**WHY THE AGENTIC ERA WILL REWARD WHOLE-SYSTEM THINKING AND
EXPOSE WEAK CONSULTING**

WHITE PAPER - SEPTEMBER 2026

NICOLÁS FERNÁNDEZ

After Agile

Why the Agentic Era Will Reward Whole-System Thinking and Expose Weak Consulting

© September 2026 Nicolás Andrés Fernández. All rights reserved.

First edition, September 2026.

ABOUT THE AUTHOR

Nicolás Andrés Fernández specializes in Microsoft Power Platform, Copilot Studio, and Dynamics 365. His contributions to the Microsoft ecosystem have earned him seven consecutive Microsoft Most Valuable Professional (MVP) awards. With more than 17 years of experience, he has designed, developed, and implemented low-code and CRM solutions for organizations across multiple industries and countries. He is the author of the Building the Center of Excellence for Power Platform book series and has extensive experience creating frameworks for adoption, governance, innovation, and sustainable platform management. Nicolás regularly shares his expertise as a speaker at Microsoft and community events, while also supporting initiatives that help organizations and professionals turn technology into sustainable business value.

COLLABORATORS

Technical reviewer: Franco Martino Gagliardi & Wilmer Alcivar

This white paper is independently published by Nicolás Andrés Fernández. It is not official guidance from Microsoft or any other vendor and does not replace official product documentation, licensing terms, security guidance, professional advice, or the organization's own legal, risk, privacy, compliance, and Responsible AI decisions. Microsoft, Power Platform, Microsoft Copilot Studio, and other product names referenced in this document are trademarks of their respective owners. References to those products are used only for identification and educational discussion.

The agentic era will reward organizations that can understand the whole, make better decisions, and turn those decisions into validated outcomes faster than those that simply produce more software.

Contents

Prologue	4
Part I: The Agile Compromise	8
1. What Waterfall Got Right, and Why It Still Failed	9
2. When Agile Became an Excuse Not to Know	12
Part II: The Economics of Understanding Have Changed	15
3. The Agentic Discontinuity.....	16
4. The Harness, Not the Model.....	19
Part III: Recovering the Whole	22
5. Before Intent: Why Are We Doing This?	23
6. The Agentic Consulting Harness	26
7. Whole-System Thinking Without Big Design Up Front.....	29
Part IV: Engineering After Agile	32
8. From Intent to Specification: Where Spec-Driven Development Fits	33
9. From Sprints to Continuous Agentic Construction	36
10. From Construction to Composition	39
Part V: The Agentic Operating Model	42
11. Consulting and Engineering as One Agentic System	43
12. The Return of Consulting	47
Conclusion - The Return of Competence	50
Continue the Series.....	53
Sources and References	54

Prologue

I began my consulting career at a time when large technology programs were predominantly shaped by plan-driven delivery models that we commonly grouped under the label Waterfall. That experience continues to influence how I look at the current debate about Agile and Agentic AI, because I have worked through both sides of the methodological shift and have seen the strengths, weaknesses, and unintended consequences of each at close range.

The older delivery models were often frustratingly slow, but they were built around an ambition that was fundamentally reasonable: before committing substantial capital to a large transformation, organizations tried to understand where they were going. They wanted to know how the future process would operate, which capabilities the business needed, which systems would participate, where information would be mastered, how integrations would work, which security principles applied, what the major dependencies were, and which decisions would be difficult or expensive to reverse later. When this worked well, it created something extremely valuable: a shared understanding of the destination that allowed business leaders, architects, delivery teams, and sponsors to discuss the same future state rather than interpreting hundreds of individual requirements independently.

The difficulty was not the value of that ambition but the amount of human effort required to realize it. Workshops generated more workshops, and requirements moved through multiple rounds of review. Architects waited for process definitions, technical designers waited for architectural direction, integration specialists waited for application decisions, and business cases waited for estimates. Each discipline depended on information produced by another, and those dependencies created long periods during which the program consumed investment without materially demonstrating progress. By the time enough analysis existed to support confident implementation, some assumptions had already changed, stakeholders had become fatigued, and sponsors were increasingly impatient to see something tangible.

Agile offered a powerful correction to this problem. It brought working solutions closer to the beginning of the lifecycle, treated feedback as an essential source of knowledge, and accepted that a complex system cannot be completely predicted before people begin interacting with it. Rather than attempting to perfect an abstract future state, teams could materialize ideas, test them against reality, learn from what happened, and adjust. Those principles remain valuable, and nothing in this paper argues that organizations should abandon them. The title should be read in the same spirit: After Agile does not mean without Agile, but after the economic constraints that shaped many of today's Agile mechanics have begun to change.

What concerns me is what sometimes happened afterward. Over time, I have seen organizations begin to treat insufficient understanding as if it were itself an Agile principle. Architecture decisions were postponed not because new evidence was required but because nobody felt prepared to make them, requirements remained vague because they could supposedly be refined later, and weak processes were automated rather than challenged because redesigning them seemed larger than the immediate sprint. Product Owners maintained sophisticated backlogs but could not always articulate why some capabilities deserved investment or what measurable outcome they were expected to influence.

I have also seen consultants who were highly competent at operating a methodology but significantly less capable of performing the underlying work of consulting. They could facilitate refinement, organize ceremonies, classify requirements, and manage delivery artifacts, yet struggle to frame the real business problem, challenge an assumption, connect a technical choice to a commercial consequence, or recommend that something should not be built. This is not a criticism of Agile itself, since Agile did not create weak consulting, weak product ownership, or weak architecture. The problem is subtler: methodology can sometimes make those weaknesses less visible because the organization continues to move. Stories are created, sprints close, demonstrations happen, velocity is reported, and the presence of motion creates confidence even when the direction remains insufficiently understood.

Agentic AI changes this discussion because it changes one of the fundamental economic constraints that shaped modern delivery. Historically, producing a sufficiently complete understanding of a large transformation was expensive because almost every analytical activity required scarce human effort. Business analysts reviewed requirements manually, architects reconciled information across applications and documents, process specialists reconstructed current-state operations, integration teams identified dependencies, security and data experts reviewed constraints, and maintaining consistency across all of those artifacts required still more people. The stronger the ambition to understand the whole, the greater the time and cost before construction could begin.

That created a rational incentive to reduce the amount of analysis performed before implementation. If understanding every major dependency required months, organizations naturally chose to understand enough to start; if some requirements were likely to change, they deferred them; and if users produced better information after seeing working software, teams started building earlier. The problem emerged when this rational response hardened into a general belief that attempting to understand more before construction was itself undesirable.

Agentic AI gives us an opportunity to revisit that conclusion, but the opportunity does not come from assuming that a generic Large Language Model can suddenly become an enterprise consultant, architect, or Product Owner. It cannot. The more significant development is our growing ability to engineer systems around these models. A well-designed harness can combine specialized agents, enterprise knowledge, consulting methods, reusable skills, architectural principles, APIs, MCP servers, platform tools, independent evaluation, persistent project context, traceability, and deliberately designed Human-in-the-Loop decision experiences. The model then becomes one cognitive component within a broader operating system that determines what information it receives, which methods it follows, what tools it can invoke, what evidence it must produce, how its output is challenged, and where human judgment must intervene.

When these elements are designed properly, the economics of understanding can change materially. Discovery that once required weeks of manual evidence collection can be parallelized, large document sets can be compared rather than sampled, requirements can be checked systematically for contradictions, and architectural alternatives can be explored at far lower cost. Process models, integration maps, decision records, technical designs, and traceability relationships can be maintained as a connected body of knowledge rather than as independent documents that gradually drift apart.

The aspiration should therefore be ambitious but disciplined. Where reliable source information exists, stakeholders are accessible, and the organization is capable of making decisions, a transformation that historically spent several months establishing its first credible design baseline should increasingly ask whether substantial parts of that baseline can be established in days or a small number of weeks. This should not be interpreted as a universal productivity promise or as a claim that six months of human work can always be compressed into five days. It is a challenge to an inherited assumption: that high-quality analysis and strong design foundations must necessarily take so long that organizations have no practical alternative but to begin building before they understand enough of the whole.

The output of this accelerated analysis should also be more than a stack of documents. A credible baseline may include a transformation blueprint, capability and process models, a Solution Design Document, functional design artifacts, Technical Design Documents, Integration Design Documents, security and data designs, architectural decisions, non-functional requirements, migration principles, deployment strategy, and the relationships that connect all of them, and its value comes not from the number of documents produced but from the coherence of the decisions they represent. Nor should those artifacts attempt to define every implementation detail, because whole-system understanding does not require complete low-level specification. A program can understand its business rationale, capability model, major process changes, target operating model, architecture, data boundaries, integration patterns, security principles, platform decisions, structural dependencies, and expensive-to-reverse choices while leaving thousands of detailed implementation decisions open to iteration.

At the same time, the downstream economics are also changing. Agents can increasingly research, plan, implement, configure, compose, test, review, remediate, and deploy technical capabilities at speeds that challenge the assumption that a one or two week sprint is the natural unit of engineering work. Platforms such as Microsoft Power Platform make this particularly visible, because managed data services, low-code application models, workflows, connectors, APIs, MCP integration, reusable skills, agents, and lifecycle tooling reduce the amount of commodity engineering that must be created from first principles.

The result is therefore neither Waterfall with AI nor Agile with a Copilot attached to every role, and it is not a claim that Spec-Driven Development, Hyper Velocity Engineering, Power Platform skills for coding agents, or any other emerging approach should replace Agile universally. It is a broader proposition: the economics of understanding and the economics of execution are changing at the same time, and if that is true, the operating model connecting them must also change.

This paper explores that possibility and its consequences for enterprise delivery, architecture, engineering, and consulting itself. It accompanies my two books, *Defining the Framework Structure for the Power Platform Center of Excellence* and *Copilot Studio and the Future of the Power Platform Center of Excellence*, in which I establish the wider CoE framework and its evolution as agents become part of enterprise technology. This paper concentrates on a narrower question: how Agentic AI may change the balance between analysis, design, iterative delivery, engineering velocity, and professional judgment. It does not argue that every program requires extensive upfront design, that Agile is obsolete, that agents eliminate uncertainty, or that faster generation automatically produces better outcomes. It asks something more

specific: what happens when organizations can afford to understand significantly more before committing, without sacrificing the adaptability that made Agile valuable?

What if the original problem with upfront thinking was not that understanding the whole had little value, but that producing that understanding cost too much and took too long?

Part I: The Agile Compromise

1. What Waterfall Got Right, and Why It Still Failed

The historical weaknesses of plan-driven delivery should not obscure one of its legitimate ambitions: major investments benefit from knowing where they are going. What frequently failed was not the desire for coherence itself, but the economics and organizational mechanics required to create and maintain it.

The legitimate ambition

A large enterprise transformation is never simply a collection of features, because it changes the way an organization works across multiple dimensions at once. Capabilities are introduced or removed, processes are redesigned, responsibilities move between teams, information changes ownership, applications acquire new boundaries, integrations redistribute dependencies, security models alter who can do what, and operating procedures evolve. In some programs, the transformation changes not only the technology estate but the architecture of work itself.

These elements are deeply interdependent. A decision about customer data can affect servicing, analytics, AI grounding, migration, security, integration, and regulatory compliance simultaneously, and a decision about process ownership can determine which application should control a transaction. A platform decision can influence whether teams are able to evolve independently in the future, while a security decision can reshape channels, identity, agent permissions, data access, and user experience at the same time. Large transformations therefore benefit from understanding the structure of the whole before every local team begins optimizing its own fragment.

Traditional design-led programs recognized this need and attempted to establish a target process model, system boundaries, architectural principles, integration approaches, data ownership, controls, and major design decisions before construction consumed the majority of the budget. When this discipline worked well, it created coherence and, equally importantly, forced consequential questions into the open. Which system is authoritative for this information? Why does this approval exist? Which capability belongs to which platform? What happens when this dependency fails? Which decisions will be expensive to reverse? Who owns this process after transformation? What assumptions are we making about future volumes or organizational behavior? These questions remain relevant regardless of methodology because they are questions about the system itself rather than about how the work is organized.

The cost of producing coherence

The problem was that producing reliable answers required large amounts of specialist human effort. Consultants interviewed stakeholders and consolidated conflicting views, business analysts translated conversations into requirements and process models, and enterprise architects collected application information from multiple owners. Solution architects waited for business analysis before defining technical direction, integration specialists investigated interfaces and dependencies, security teams reviewed policies and risks, and data architects reconciled inconsistent concepts across systems.

Every activity generated new dependencies. One team discovered a contradiction and requested another workshop, a platform limitation invalidated an earlier process assumption, and a regulatory clarification reopened a design decision. An unresolved ownership question delayed integration design, while a change in architecture forced updates to functional and technical documents. Although the work was intellectually connected, it was operationally fragmented, which meant that even competent teams produced long elapsed times because information moved through organizational queues.

Sponsors therefore saw expenditure well before they saw materialized value, and stakeholders participated in repeated workshops without experiencing the future solution. Delivery teams spent months producing confidence on paper before the organization had enough working software to challenge that confidence, so that by the time the analysis reached an acceptable level of completeness, parts of the original context could already have changed.

When analysis stops helping

There is another uncomfortable aspect of this history: additional analysis does not always produce additional clarity, and sometimes it simply delays commitment. A consultant who lacks confidence can request more information instead of making a recommendation. An architect can continue evaluating options because selecting one requires acknowledging that every option has disadvantages. A stakeholder can postpone accountability by requesting another scenario, while a Product Owner can continue gathering requests because defining what matters is harder than documenting what people ask for.

In these situations, the problem is no longer insufficient information but insufficient judgment. Analysis becomes a mechanism for avoiding the decision it was supposed to support, and rigor gradually turns into delay. This is one reason older methodologies created so much frustration: organizations could spend months producing artifacts without proportionally improving their ability to decide.

The lesson should have been that analysis must remain tightly connected to decision-making and that a design process should stop when additional investigation no longer changes the decision. Instead, some organizations learned a broader lesson and concluded that upfront analysis itself was wasteful, a conclusion that removed an important distinction between excessive analysis and useful whole-system understanding.

Agile entered this environment with a compelling alternative. If requirements will change, avoid overinvesting in requirements that may be invalidated; if users understand the solution more clearly once they can interact with it, let them interact sooner. If architecture cannot predict every future need, allow some detail to evolve, and if working software provides stronger evidence than documentation, build earlier. These ideas remain sound. The problem appears when the economic difficulty of obtaining a whole-system view becomes confused with evidence that such a view has little value.

SPOTLIGHT

The problem with upfront thinking was not the value of having a coherent target vision. The problem was that producing and maintaining that vision was often prohibitively slow, expensive, and dependent on scarce human capability.

Whole-system thinking is not omniscience

A coherent vision does not require every detail to be known, and this is where many methodological debates become unnecessarily polarized. The choice is not between a completely specified system and no meaningful upfront understanding. A program can know its strategic objectives, business capabilities, major process changes, operating model, application architecture, data domains, integration principles, security model, platform boundaries, and critical dependencies without deciding every screen, field, rule, prompt, workflow step, or interface behavior.

The correct question is therefore not whether everything can be predicted, because it cannot. The useful question is which decisions require knowledge of the whole before local implementation begins and which decisions genuinely benefit from being left open until evidence becomes available. Once framed in this way, whole-system thinking and adaptive delivery stop looking like ideological opposites and become complementary techniques applied at different levels of the same problem.

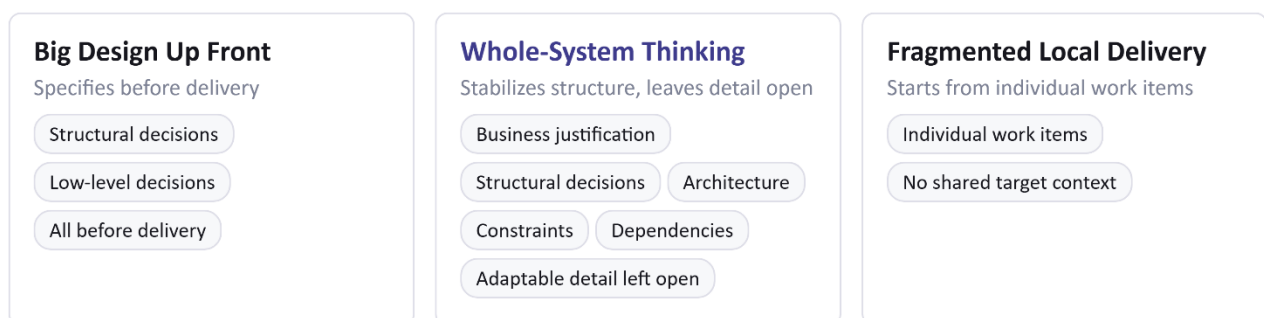


Figure 1. Three ways to approach the whole.

2. When Agile Became an Excuse Not to Know

Agile solved a genuine problem by recognizing that working solutions generate knowledge that planning cannot. Its misuse begins when uncertainty that could reasonably be reduced through analysis is treated as if only implementation can resolve it.

What Agile corrected

Agile's most durable contribution is not a ceremony, role, or artifact but a way of thinking about knowledge: reality contains information that plans do not. A customer may react differently from what research predicts, an employee may resist a workflow that appears efficient in a process model, a regulatory interpretation can evolve, a technical component can behave differently at scale, and an AI agent can perform well in a controlled evaluation while failing when exposed to production variability. No amount of documentation can eliminate these uncertainties because the evidence does not yet exist.

The appropriate response in such situations is to create that evidence by materializing the idea, observing behavior, measuring the result, and adapting, a response that remains essential in the agentic era. The original Agile principles also make clear that agility was never intended to oppose engineering discipline, since technical excellence and good design are presented as contributors to agility rather than as alternatives to it. The important tension is therefore not Agile versus design, but genuine uncertainty versus avoidable uncertainty.

Genuine and avoidable uncertainty

Some questions cannot reasonably be answered without real-world interaction. Will customers adopt a new interaction model? Will employees trust recommendations generated by an agent? Will a new commercial proposition change buying behavior? Will an AI system maintain sufficient quality when exposed to production variation? These are examples of genuine uncertainty because reality itself is the source of the missing evidence.

Other questions are fundamentally different. What interface does the legacy system expose? Where is customer information mastered? Which regulation applies to this transaction? Which team owns the approval? Do two requirements contradict each other? Does the existing platform already provide the requested capability? Which architectural alternative creates greater vendor lock-in? These questions may be difficult and may require expertise, research, or organizational negotiation, but they are not inherently unknowable. They are often simply uninvestigated.

This distinction matters because poor delivery models sometimes treat both categories identically, so that everything becomes something that can be discovered later, even when the uncertainty could have been reduced before implementation at comparatively low cost. Historically, organizations had a legitimate economic reason for doing this, because investigation itself was expensive. If analyzing an integration landscape required several specialists for multiple weeks, the program might reasonably choose to begin

building the most visible capabilities instead. If comparing three architectural options required three separate prototypes, evaluating only one option might appear proportionate, and if reviewing thousands of requirements required a large team, some inconsistencies would inevitably survive into delivery.

Agile was capable of absorbing these realities because it expected learning to continue after implementation had started, and that flexibility was useful. The problem appeared when the economic necessity disappeared from the explanation while the behavior remained, and “we can discover it later” became a default response even when the question could have been answered economically beforehand.

When the backlog replaces the system

A backlog is very effective at organizing work but much less reliable as a representation of the whole system. An Epic can make sense, the Features beneath it can be reasonable, each User Story can have clear acceptance criteria, and every task can be executable, while the combined result remains architecturally incoherent or strategically unjustified.

Decomposition is not understanding. Breaking work into smaller pieces does not prove that the organization understands how those pieces fit together, and this becomes particularly dangerous because Agile artifacts create an appearance of order. The backlog is prioritized, stories are refined, dependencies are recorded, and progress is visible, yet the organization may still be unable to answer a more fundamental question: what coherent future state do all of these items collectively create?

The same weakness appears in product management when a Product Owner can decide confidently which story should be delivered next but cannot connect that story to a measurable business outcome or explain the value hypothesis behind it. That is not a failure of the backlog itself but a failure of the context above the backlog.

Methodology in general has enormous value because it provides structure, but structure can create false confidence when it becomes a substitute for thought. A program can look healthy because sprints close, demos happen, velocity is reported, and work items move through workflow states, so the organization experiences motion even though motion is not necessarily progress. A Waterfall program could produce hundreds of pages without solving the right problem, just as an Agile program can produce hundreds of stories without creating the right system. The failure mode changes shape, but the underlying issue is similar: methodology cannot substitute for judgment.

Resolve uncertainty at the cheapest responsible point

The cost of an unresolved question generally increases as it moves downstream. If a process misunderstanding is discovered during initial analysis, resolving it may require a conversation; if the same misunderstanding is discovered after configuration, it creates rework. If an architectural incompatibility is identified while comparing options, the correction may require hours; if multiple teams have already implemented against the wrong assumption, the same correction may require months. The objective should therefore not be to move every decision upfront, but to resolve each uncertainty at the earliest point at

which sufficient evidence exists and at which the cost of resolution is lower than the cost of carrying that uncertainty forward.

Do not iterate over what you could have understood. Iterate over what can only be learned through experience.

Agile should adapt to the agentic era

Agentic AI changes the environment in which Agile operates, and this should force us to reconsider which current practices are responses to enduring principles and which are responses to historical constraints. If research becomes cheaper, the economically rational amount of research changes, and if prototypes become inexpensive enough that several alternatives can be tested before commitment, architectural exploration changes. If implementation becomes dramatically faster, the natural technical cadence changes. If agents can dynamically generate and revise implementation plans, the backlog may no longer need to represent every technical step permanently.

The genuinely Agile response is therefore not to defend today's Agile mechanics because they are familiar, but to inspect the new reality and adapt the operating model accordingly.

Part II: The Economics of Understanding Have Changed

3. The Agentic Discontinuity

The most consequential effect of Agentic AI is not faster code generation but the possibility of reducing the cost of research, analysis, design, experimentation, verification, and execution at the same time, thereby changing how much uncertainty an organization can economically remove before commitment.

Beyond the coding assistant

The first generation of Generative AI in engineering was naturally interpreted as individual productivity. A developer asks for code, a model produces a draft, the developer reviews and adjusts it, and the task takes less time. This is useful, but the surrounding delivery model remains almost unchanged because the analyst still analyzes manually, the architect waits for inputs, the Product Owner organizes the backlog, the tester receives implementation at the usual point, and the developer is simply faster.

Agentic systems create a more consequential possibility because they can participate across workflows rather than inside isolated tasks. One agent can retrieve and structure evidence while another compares requirements, another inspects process documentation, another analyzes architecture, another produces an implementation plan, another creates the solution, and another independently tests the result against explicit criteria. The system can call tools, query repositories, invoke APIs, use MCP servers, preserve project state, and propagate changes between connected artifacts, so the transformation moves from task acceleration toward redesign of the knowledge-to-value chain itself.

Upstream compression

The first discontinuity appears before construction, where a major transformation may depend on hundreds or thousands of information sources, including strategy documents, process maps, application inventories, policies, regulatory materials, data models, integration catalogs, legacy requirements, architecture decisions, workshop outputs, and operational evidence. Human teams process this information incrementally, often discovering a contradiction only after one document has already informed several downstream decisions.

An agentic system can process significant portions of this evidence in parallel, search across large corpora, compare statements systematically, maintain relationships between findings, identify missing evidence, generate multiple architectural alternatives, and update downstream artifacts when assumptions change. This does not mean that the system understands the enterprise autonomously; it means that much of the mechanical work required to prepare evidence for human understanding can become cheaper and faster, which is what this paper calls upstream compression. Where evidence exists, the problem is sufficiently bounded, and decision-makers are available, analysis that historically consumed months should increasingly be challenged to establish its first credible baseline in days or a small number of weeks.

WARNING

“Months to days” should never be treated as a universal productivity ratio. Agentic systems can accelerate research, synthesis, comparison, artifact creation, option analysis, and verification, but they cannot automatically resolve political disagreement, missing information, unavailable stakeholders, regulatory ambiguity, or decisions that nobody is willing to own. Treating those human and organizational constraints as if they were merely computational would replace methodological optimism with AI optimism.

Better foundations rather than faster paperwork

The purpose of upstream compression is not simply to reproduce the traditional analysis process at a higher speed, because doing so would only allow organizations to generate the same documentation more quickly without necessarily improving the decisions behind it. The more important opportunity is to strengthen the analytical foundation itself by examining more evidence, exposing contradictions earlier, considering a broader range of alternatives, and preserving relationships between business objectives, capabilities, architecture, and downstream design decisions that human teams historically struggled to maintain consistently.

A well-designed agentic discovery environment should therefore make changes easier to understand across the system. If a business goal changes, the organization should be able to identify which capabilities are affected, and if a capability changes, the relevant processes and architectural components should become visible. If an architectural decision is reconsidered, dependent specifications and designs should be flagged; if a security constraint changes, affected integrations and technical choices should be surfaced before implementation diverges from the new rule. The real value of compression is therefore not the production of more material in less time but the ability to reach a coherent and defensible understanding of the system sooner, while keeping that understanding synchronized as decisions evolve.

Downstream compression

The second discontinuity occurs during construction, where agentic systems increasingly support research, planning, generation, configuration, testing, review, remediation, documentation, and deployment. Managed platforms reduce the required construction effort further by providing identity, persistence, workflow, integration, observability, governance, and lifecycle services as reusable capabilities.

As these capabilities converge, work that previously occupied a substantial portion of a sprint may move through several technical iterations within a single day. This is downstream compression, but once again the value should not be confused with speed alone, since a faster engineering system fed by poor decisions merely produces high-velocity rework.

The two compressions are therefore interdependent, and the transformation becomes meaningful when upstream and downstream change together. Fast analysis combined with traditional construction can produce a better blueprint but retain much of the old delivery economics, while fast construction combined with weak analysis can produce the wrong solution with remarkable efficiency. The more interesting

operating model emerges when stronger upstream understanding is combined with governed, rapid implementation and continuous verification.

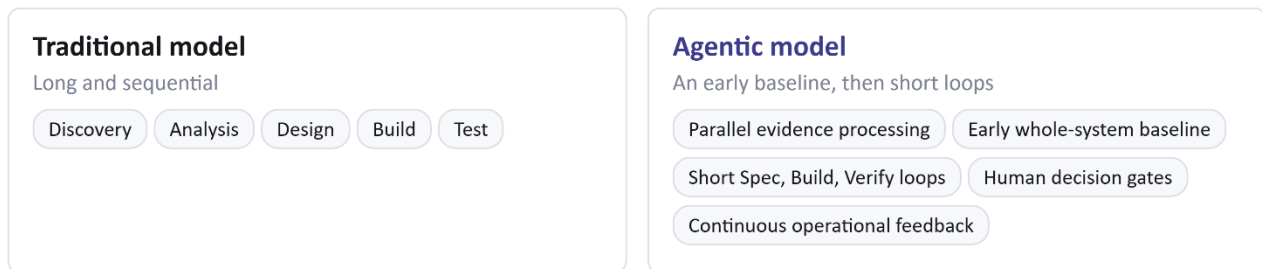


Figure 2. Traditional and agentic delivery models.

The economic frontier of knowability

This suggests a useful concept: the economic frontier of knowability. At any point in a program, there are questions whose answers are theoretically available but too expensive to obtain. Traditional delivery operated with a relatively narrow frontier because every additional investigation consumed significant human time, so organizations rationally accepted uncertainty even when the answer could, in principle, have been found.

Agentic systems can move that frontier. A question that previously required several specialists for two weeks may now require a few hours of machine-supported preparation followed by focused expert review, an architecture comparison that previously justified one prototype may support three or four alternatives, and a requirements analysis that previously sampled representative cases may inspect the entire corpus.

The system still cannot know the unknowable; what changes is the amount of the knowable that becomes economically practical to know. That is the agentic discontinuity that matters for the operating model.

4. The Harness, Not the Model

A powerful model can generate sophisticated output, but reliable enterprise capability depends on the operating system surrounding that model: context, method, tools, evidence, permissions, verification, and human decision rights determine whether model capability becomes organizational capability.

Why model capability alone is insufficient

Modern models are persuasive because they can discuss strategy, generate process models, critique architecture, draft requirements, propose integration patterns, and produce technical documents with remarkable fluency. That fluency creates a risk because the model can sound as though it understands the organization even when it lacks the context required to make a responsible decision.

It may not know which internal source is authoritative, may treat an outdated architecture diagram as current, may not understand which policy has precedence, or may fail to distinguish a binding requirement from a historical convention. It may answer confidently when the responsible action is to request evidence or escalate a decision, and it may review its own output using the same assumptions that created the mistake in the first place. This is why the central architectural concept is not the model but the harness, meaning the surrounding system that structures how model capability is used and how its output becomes trustworthy enough to influence real decisions.

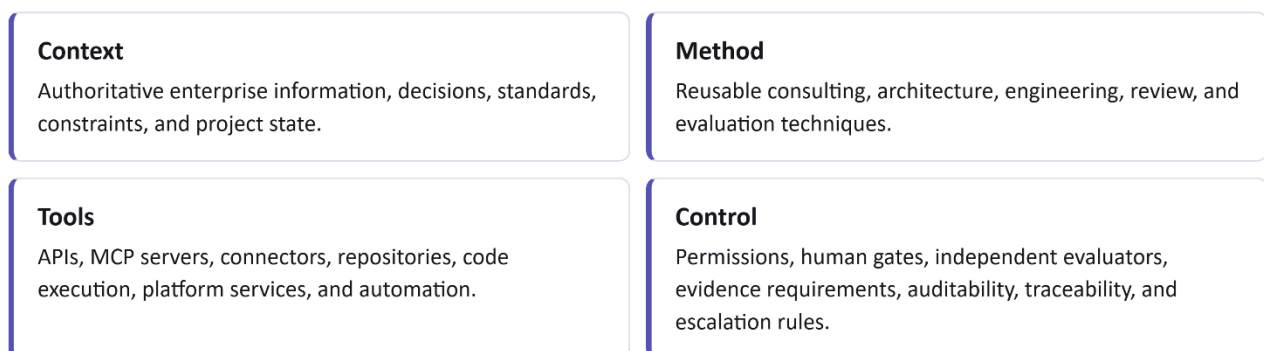


Figure 3. Four components of an agentic harness.

The harness may be implemented across several technologies and does not need to exist as a single monolithic product. What matters is that the organization deliberately engineers the environment in which the model operates instead of assuming that access to a powerful model is equivalent to possessing an enterprise capability.

Quality becomes systemic

DORA's 2025 research on AI-assisted software development provides an important signal because it shows that AI does not automatically improve delivery. Its impact depends heavily on the organizational system in which it operates, meaning that strong practices can be amplified while weak practices can also be amplified, and this observation supports a broader principle for agentic work.

In the agentic era, quality becomes a property of the harness, not merely a property of the model.

Two organizations can use the same underlying model and possess radically different capabilities. One may provide a prompt and access to several documents, while another provides validated enterprise context, architecture principles, reusable methods, specialist skills, approved tools, independent reviewers, explicit decision rights, and traceability. The model may be identical, but the resulting operating capability is not, and as access to high-quality models becomes less differentiating, the system surrounding them becomes a more important source of competitive advantage.

Expertise becomes partially reusable

This has significant consequences for professional services because expertise has historically been consumed primarily through time. A senior integration architect knew how to evaluate interfaces, a process expert knew how to challenge a workflow, a security specialist knew which evidence mattered, and a strong consultant knew which requirement sounded reasonable but did not survive scrutiny. The organization accessed that expertise when the expert personally performed the work.

A harness allows parts of that expertise to become reusable infrastructure. The architect's assessment method can become a structured skill, security review criteria can become an evaluator, a discovery approach can become an agentic workflow, and a requirements challenge can systematically search for missing actors, unsupported assumptions, conflicting goals, and unclear outcomes.

This does not capture the entirety of professional judgment, because judgment depends on context, experience, accountability, and the ability to reason through novel situations. It does, however, reduce the amount of scarce expert time consumed by repeatable analytical work and allows specialists to concentrate on exceptions, ambiguity, and consequential trade-offs.

That reuse is not free. An encoded method is a living asset with an owner, a review cycle, and a decay rate, and when the platform, the regulation, or the practice changes, a skill nobody maintains goes on applying last year's judgment at machine speed.

The factory becomes a knowledge system

This is where the idea of an Agentic Factory becomes useful, provided the term is not interpreted as a software factory designed merely to maximize output. Where a harness equips one line of work, the factory is the organization's shared stock of harness components: the methods, skills, evaluators, and rules from which each harness is assembled. Its purpose should be to industrialize repeatable quality by turning methodologies, reference architectures, skills, decision models, integration patterns, evaluation criteria, test assets, and governance rules into reusable organizational capabilities.

The economic model therefore begins to move away from expertise multiplied only by human hours. A more scalable model combines expertise with reusable systems and then concentrates human judgment on

decisions where it creates disproportionate value, and this shift becomes particularly important once the discussion moves from engineering productivity into the economics of consulting itself.

Part III: Recovering the Whole

5. Before Intent: Why Are We Doing This?

Intent is essential for engineering execution, but it is not a business justification. A credible agentic operating model therefore needs a hierarchy above intent that explains why the organization should act, what outcome it expects, and which capabilities are required before technology begins optimizing implementation.

Intent starts too low for transformation

AI-native engineering increasingly speaks about intent, and that emphasis is understandable because an agent needs to know what it is trying to accomplish, a specification needs a desired behavior, and an implementation needs acceptance criteria. In a large transformation, however, “What should this component do?” is not the first intellectual question. The first question is why the organization should change at all.

A useful hierarchy therefore begins with motivation, which describes the condition creating pressure or opportunity and explains why action is being considered now. Operating costs may be unsustainable, customer satisfaction may be deteriorating, a regulatory requirement may be approaching, a legacy platform may be reaching end of life, a competitor may have changed market expectations, or a new commercial opportunity may have appeared.

The strategic objective then translates that pressure into a direction, such as reducing cost-to-serve while maintaining service quality, increasing digital conversion, improving operational resilience, reducing onboarding time, or increasing employee productivity. Importantly, the objective still does not prescribe a solution, and that separation protects the organization from confusing its desired business direction with the first technological intervention that happens to be proposed.

Goals, outcomes, and value hypotheses

A goal makes the strategic ambition measurable: an organization may want to reduce handling time by 30 percent, increase digital conversion by ten percentage points, reduce manual processing by half, or reach a defined reliability target. The expected outcome then describes the observable change that should help produce that goal, such as fewer repeat contacts, higher first-contact resolution, more straight-through processing, less manual reconciliation, or faster onboarding.

The value hypothesis connects intervention to outcome by explaining why the organization believes a particular change will create the desired effect. If service advisors receive relevant customer context and approved knowledge during an interaction, search time should fall and first-contact resolution should improve. If document validation becomes automated, processing time and manual workload should decrease. If customer context becomes unified across channels, service consistency and conversion should improve.

The hypothesis is important because it can be wrong. Once it is explicit, operational evidence can challenge it, so that if the outcome does not materialize, the organization can question the logic behind the investment rather than simply adding more features to the same solution.

Business capabilities as the bridge

Business capabilities describe what the organization must become able to do in order to create those outcomes. Real-time customer identification, intelligent case triage, contextual knowledge retrieval, automated document validation, and proactive communication are examples, because they remain oriented around business ability without prematurely choosing technology.

Capabilities therefore create a bridge between strategic direction and architecture. Once the required capabilities are sufficiently understood, the organization can begin shaping the whole-system vision that determines how processes, people, data, applications, integrations, controls, and technologies should combine to deliver them, and only after this context exists does downstream intent become meaningful.

At that point, an intent describes what a particular system, agent, application, or component should accomplish within the wider context, while a specification makes that intent precise enough to plan, implement, and verify. The order matters because a technically excellent specification cannot compensate for weak business justification.



Figure 4. The chain of justification, from motivation to evidence.

A running example

Consider a service organization facing rising operating costs and declining customer satisfaction. Its motivation is unsustainable service economics, while its strategic objective is to reduce cost-to-serve without degrading quality. A measurable goal might be to reduce average handling time by 25 percent while increasing first-contact resolution, and the expected outcomes could include fewer transfers, less manual information search, and more consistent responses.

A value hypothesis might state that giving advisors relevant customer context and approved knowledge at the point of interaction will reduce search time and prevent repeated contact. The business capabilities required could therefore include unified customer context, intelligent triage, contextual knowledge retrieval, and assisted case handling.

Only after this logic has been established does a downstream intent such as “Provide the advisor with the most relevant approved knowledge for the current case and customer context” become meaningful. The intent now inherits business justification rather than existing as an isolated instruction.

Consulting explains why

This distinction creates one of the most important boundaries in the model, because a team can implement a specification perfectly and still create no meaningful value.

A specification explains what a solution must do. Consulting explains why that solution deserves to exist.

Such a team can automate an approval that should have been eliminated, implement a requirement that exists only because of a legacy constraint, optimize a local activity while damaging the end-to-end journey, or solve the stated problem while missing the underlying one. No amount of downstream engineering quality compensates for poor upstream framing.

Reengineering before automation

This becomes more important, rather than less, when construction becomes cheap. Slow implementation naturally creates friction, and that friction sometimes prevents weak ideas from immediately becoming software. Agentic construction removes part of that friction, which means that a badly framed requirement can become a working solution very quickly.

The consulting layer therefore has to become more challenging. It must ask whether the process should exist, whether a control is genuinely required, why a handoff occurs, whether the future process would look different if the legacy application were not treated as a constraint, whether Agentic AI changes the operating model rather than merely improving the interface, and what the organization should stop doing altogether. These are not implementation questions but questions of problem framing, value, and organizational design, which is precisely why consulting remains essential in an increasingly automated delivery environment.

6. The Agentic Consulting Harness

The Agentic Consulting Harness uses machine-scale analysis to improve human decision-making. Its purpose is not to automate judgment away, but to make high-quality consulting more evidence-rich, more repeatable, and less constrained by the manual cost of analysis.

From document production to decision preparation

Traditional consulting consumes significant effort preparing information for decisions: teams collect documents, summarize interviews, map processes, reconcile inventories, classify requirements, build option matrices, update presentations, and trace dependencies. This work matters because decisions require evidence, but much of it is preparatory rather than judgment-intensive.

An Agentic Consulting Harness can redistribute that workload. A process agent can compare current-state variants, an enterprise-architecture agent can analyze application portfolios, a data agent can identify inconsistent ownership assumptions, a security agent can evaluate policy constraints, a business-analysis agent can reconcile stakeholder requirements, a challenger agent can deliberately search for contradictions and unsupported claims, and a financial-analysis agent can model alternative scenarios. The purpose is not to let those agents make unaccountable strategic decisions. It is to provide the human professionals who remain responsible for those decisions with a richer, more coherent evidence base than they could economically assemble through manual work alone.

Parallelism changes elapsed time

Much traditional analysis is slow because disciplines operate in sequence. The solution architect waits for business analysis, the integration architect waits for application boundaries, the security team waits for architecture, and the financial model waits for cost estimates, so even when every team works efficiently, the elapsed time remains high because the dependency chain is long.

An agentic system can perform significant portions of this preparation in parallel. While one agent analyzes processes, another can inspect applications, another can review policies, another can identify unresolved decisions, and another can update the capability model. As evidence changes, dependent analyses can be regenerated rather than manually reconciled through another cycle of meetings and document updates. The important improvement is therefore not simply faster writing but shorter elapsed time between question and decision.

A credible baseline, not an illusion of completeness

The goal of the Consulting Harness should be a credible baseline rather than an illusion of exhaustive certainty. A multinational transformation may still require considerable time because stakeholder alignment can remain difficult, politics remain real, source information can be incomplete, and regulatory interpretation can remain ambiguous. Agentic capability does not remove these constraints.

What should change is the expectation surrounding the analytical work that exists around them. If a program historically allocated three or six months before it could produce a sufficiently coherent blueprint and architecture baseline, it should now challenge whether properly designed agentic support can establish substantial portions of that foundation much earlier. The output should be decision-grade coherence: enough understanding to support the next level of commitment while clearly distinguishing resolved questions from those that remain genuinely uncertain.

Human-in-the-Loop as decision augmentation

The design of Human-in-the-Loop becomes critical because human involvement is not valuable merely because a person clicks an approval button. A poor implementation can generate a long document, present it to a time-constrained executive, receive an approval, and technically claim human oversight without improving the quality of the decision.

A stronger implementation concentrates the relevant information around the decision itself. Imagine three architecture alternatives presented against common dimensions such as cost, security, operational complexity, implementation difficulty, vendor dependency, reversibility, time-to-value, and known assumptions. The decision-maker can change a constraint and immediately explore what happens if volume doubles, one legacy system is retired earlier, or a regulatory assumption changes, while the system can expose the evidence supporting its estimates and generate an additional option if the existing alternatives remain unsatisfactory. The human then chooses, rejects, or asks for further analysis, and the rationale becomes part of persistent project context that constrains downstream design.

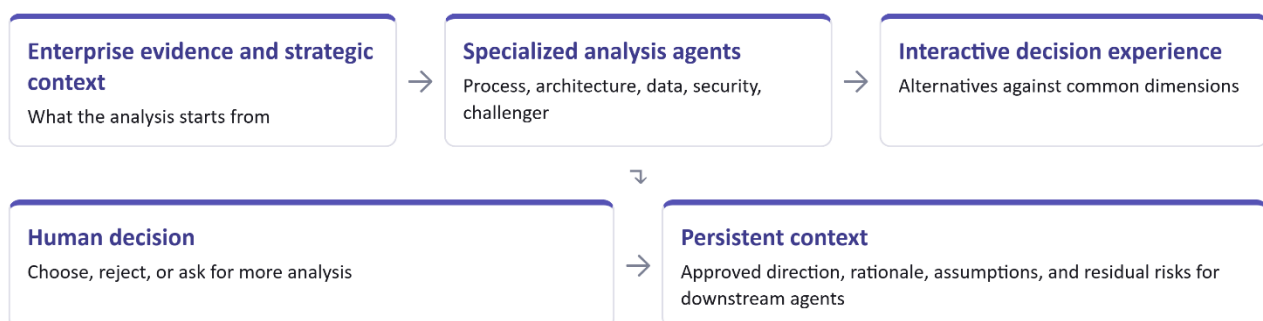


Figure 5. Human-in-the-Loop as decision augmentation.

This is HITL as decision augmentation. The goal is not to minimize human participation, but to use human attention where it has the greatest value.

Some decisions also involve responsibilities that should not be transferred to an agent simply because the agent can produce a recommendation. Strategic investment, material risk acceptance, regulatory interpretation, major architecture lock-in, operating-model change, workforce consequences, and ethical boundaries remain organizational responsibilities, and while agents can improve the evidence, expose inconsistencies, model scenarios, and challenge assumptions, accountability remains with people. This is not a weakness of the operating model but an essential design principle, because decision rights and responsibility should remain aligned.

Better consulting rather than automated consulting

The strongest form of Agentic Consulting is therefore not autonomous consulting. It is amplified judgment: a good consultant with a strong harness can examine more evidence, test more alternatives, maintain better traceability, and spend more time on difficult questions than the same consultant working manually.

A weak consultant with the same tools can also generate much more material. That asymmetry becomes important because Agentic AI does not merely increase productivity; it also makes the difference between producing consulting artifacts and possessing consulting judgment much more visible.

7. Whole-System Thinking Without Big Design Up Front

Whole-system thinking establishes enough structural clarity to keep local execution coherent while deliberately leaving implementation detail open where evidence and iteration add value.

What must be understood

A credible initial baseline for a large transformation should answer structural questions about why the program exists, which measurable outcomes justify the investment, which capabilities must change, which processes require redesign, what operating-model implications exist, where the major application boundaries should lie, where important information is mastered, how systems should integrate, how identity and security should operate, which non-functional constraints matter, and which platform choices will shape the future system.

It should also make major dependencies visible, identify decisions that will be expensive to reverse, and distinguish genuine unresolved uncertainty from questions that simply have not yet been investigated. If the program cannot answer these questions at all, it does not yet possess a meaningful view of the whole.

The design baseline

The exact artifacts will vary by organization, but a large transformation may require a connected family of outputs such as a transformation blueprint, business capability model, target process model, target operating model, Solution Design Document (SDD), functional design, Technical Design Documents, Integration Design Documents, data architecture, security and identity design, environment strategy, non-functional requirements, architectural decisions, migration principles, test strategy, and delivery dependencies.

The acronym is not the important part; the relationship between the artifacts is. A beautifully written SDD that has no connection to the business capability model is not a coherent design, a technically detailed TDD based on an unresolved architectural decision is premature, and an IDD that describes an interface perfectly but cannot explain why the integration exists is locally precise and strategically weak. The quality of the design baseline therefore depends less on document completeness than on whether the documents express a consistent model of the same future system.

The maintenance problem

One of the oldest problems in enterprise design is not producing documentation but keeping documentation true. A business process changes and the functional design is updated, while the architecture remains unchanged. A platform constraint invalidates an integration assumption, so the interface design changes, but the test strategy still reflects the previous version. After several months, every document describes a slightly different system. This is one area where agentic tooling can materially change

the economics, because artifacts can increasingly be treated as different views over a connected model rather than independent files that must be manually reconciled whenever a decision changes.

The Living Design Graph

The target should therefore be a Living Design Graph in which motivation connects to objectives, objectives to goals, goals to outcomes, outcomes to value hypotheses, value hypotheses to capabilities, capabilities to processes, processes to architecture, architecture to intent, intent to specifications, specifications to implementation, implementation to validation, and validation to operational evidence.

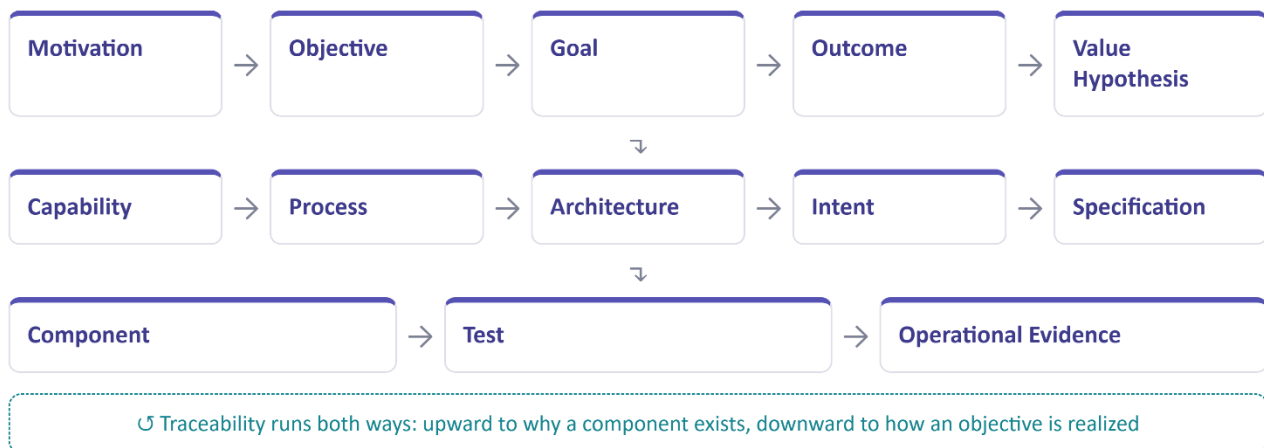


Figure 6. The Living Design Graph: justification carried into design.

Figure 6 is not a second version of Figure 4. Figure 4 is the chain of justification: it explains why something should be built, and it compresses everything between capability and intent into a single whole-system vision. Figure 6 is the design model that keeps that justification traceable once work begins, opening the whole-system vision into the processes and architecture that realize each capability and ending not in a generic implementation but in the components and tests that can actually be traced. Architecture comes before intent in both because intent, as defined in Chapter 5, describes what a particular system, agent, or component should accomplish, and which components exist and what each is responsible for is an architectural decision. An intent written before the architecture has no component to belong to, which is also why, in continuous improvement, intent can enter low in the model: the architecture it depends on already exists.

This creates Strategic-to-Execution Traceability. Someone examining a technical component should be able to follow the reasoning upward and understand why it exists, while someone examining a strategic objective should be able to move downward and see how it is being materialized through capabilities, architecture, specifications, and implementation. That is a stronger concept than traditional requirements traceability because it preserves business justification, not merely functional lineage.

What should remain flexible

The existence of a whole-system baseline does not mean that local implementation is frozen. Thousands of decisions can remain open, including user-interface details, workflow refinements, field-level validation, reports, prompt wording, agent instructions, low-risk business rules, and detailed error handling.

These decisions are exactly where iteration can create value, because their consequences are often local, reversible, or dependent on user evidence. The principle is therefore not to stabilize everything but to stabilize what requires coherence and preserve flexibility where learning is beneficial.

Maximum Economically Rational Clarity

This leads to the central design principle of the paper: Maximum Economically Rational Clarity. The objective is not maximum possible clarity, because there will always be another question, another scenario, another edge case, and another analysis, and attempting to remove all uncertainty recreates the old trap in which analysis becomes disconnected from decision-making.

The objective is instead to resolve what can be resolved economically and to preserve uncertainty where further evidence genuinely requires experimentation or real-world use. If an important unknown can be answered cheaply through research, it should be answered. If an architectural hypothesis can be tested with an inexpensive prototype, it should be tested. If only operational use can produce credible evidence, the uncertainty should remain explicit and be managed through iteration. If the true obstacle is an unresolved executive decision, it should be escalated rather than disguised as a need for additional analysis.

As the marginal cost of understanding falls, the rational amount of understanding should increase, although analysis should still stop when the cost of additional certainty exceeds the risk that certainty would remove.

SPOTLIGHT

Whole-system thinking does not require complete low-level specification, and agility at the detail level does not require ambiguity at the system level.

Part IV: Engineering After Agile

8. From Intent to Specification: Where Spec-Driven Development Fits

Spec-Driven Development is a valuable engineering discipline for preserving intent through implementation. It becomes especially powerful when the surrounding business and architectural context already exists and can be inherited by the specification.

Why specification is returning

One of the most revealing changes in AI-native engineering is the renewed emphasis on specification. For years, detailed specification became culturally associated with heavyweight delivery, which led some Agile environments to treat substantial upfront definition with suspicion. Agentic engineering changes the incentive, because the faster an implementation can be produced, the more costly ambiguous intent becomes. A human development team may spend days gradually discovering that a requirement has been misunderstood, while an agentic system can materialize that misunderstanding almost immediately. Faster execution therefore increases the value of precise direction.

Spec-Driven Development reflects this shift by treating structured specification as shared context connecting intent, architecture, implementation, testing, and validation. The important point is not a return to static documentation, since a modern specification can evolve as evidence changes. What matters is that intent remains explicit, persistent, and available to the engineering system rather than being reconstructed from fragmented conversations and backlog items.

What Spec Kit contributes

GitHub Spec Kit provides a practical example of this approach through a workflow that includes project constitution, specification, clarification, planning, quality checks, task decomposition, analysis, implementation, and convergence. The constitution establishes project-level principles, the specification concentrates on desired behavior and goals, planning determines how the capability should be implemented, and convergence compares the resulting implementation with the specification and plan.

This separation of abstraction is valuable because it prevents one work item from having to carry every type of information: strategic and project-level constraints can remain persistent while an individual specification brings one unit of intent to the precision necessary for reliable implementation.

It is important to be precise here, because Spec-Driven Development does not inherently propose that individual specifications should exist in isolation. Spec Kit includes project-level principles, and its Spec of Specs pattern explicitly introduces a decomposition step when a problem is too large for one specification cycle, which demonstrates that specification-driven engineering can preserve wider context while still allowing local implementation units to remain manageable.

There is still an important boundary, however. A project constitution is not automatically a business strategy, and a Spec of Specs is not necessarily a transformation blueprint; a specification can preserve engineering intent extremely well without proving that the intent represents the right business investment. This is why Spec-Driven Development fits inside the Agentic Operating Model rather than replacing the upstream consulting layer.

Continuous Improvement Mode

Spec-Driven Development is especially natural when the whole already exists. Consider a mature product whose strategic purpose is known, whose architecture is established, whose data ownership is clear, whose security principles exist, whose operational constraints are understood, and whose main capability boundaries are sufficiently stable.

Return to the service organization from Chapter 5 once its advisor-assistance capability is live. A new intent such as “Improve prioritization of service cases using customer context and urgency” can now inherit the surrounding context: the handling-time goal, the unified customer context, and the architecture that already provides it. The organization does not need to redesign its entire operating model before evaluating the change; it can assess the intent against existing goals and architecture, convert it into a specification, plan the implementation, build it, verify the result, and learn from operational evidence.

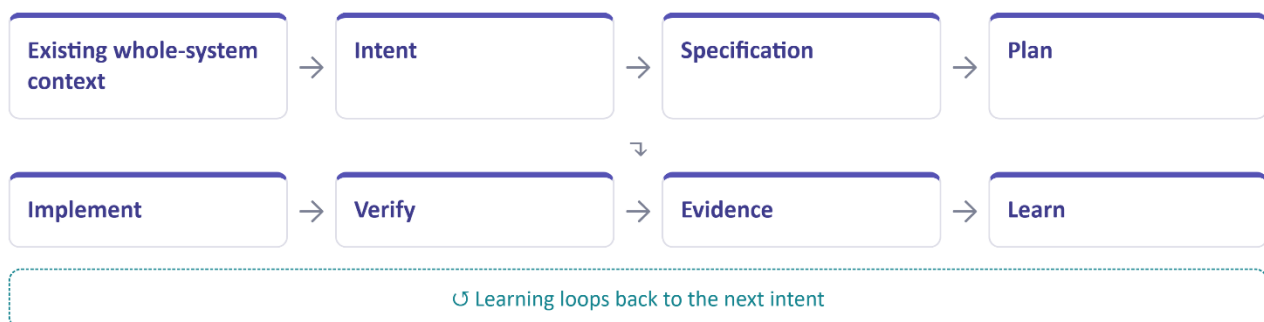


Figure 7. In continuous improvement, intent inherits the whole.

The principle is straightforward: in continuous improvement, intent can inherit the whole.

Transformation Mode

A major transformation is different because the whole may not yet exist. If the organization is redesigning customer service across countries, replacing a fragmented legacy estate, changing the operating model, and introducing agents into core processes, then individual intents depend on upstream choices that have not yet been made.

The organization first needs sufficient clarity about business objectives, measurable goals, capabilities, process design, data, integration, architecture, security, platform boundaries, and structural dependencies. Only after those elements become coherent can local intents inherit a meaningful system context, which is why the corresponding principle is that in transformation, the whole must often be established before meaningful intents can be defined.

Specification Myopia

This distinction points to a potential failure mode that should not be confused with a defect inherent to Spec-Driven Development. Agile organizations sometimes transformed the User Story from a collaboration mechanism into the sovereign unit of reality, becoming excellent at optimizing stories while losing visibility of the whole system, and a similar organizational pattern could emerge around specifications.

A specification can be excellent, the plan can be rigorous, the implementation can follow the plan precisely, and every test can pass while the resulting capability still represents the wrong investment. I call that risk Specification Myopia, meaning the optimization of individual specifications without preserving adequate traceability to outcomes, capabilities, architecture, and strategic justification.

WARNING

The risk is not Spec-Driven Development. The risk is treating specification quality as a substitute for strategic coherence. A perfectly engineered specification can still be the specification of the wrong thing.

9. From Sprints to Continuous Agentic Construction

Agentic engineering does not necessarily reduce iteration. It can increase the frequency of technical iteration so dramatically that the sprint becomes primarily a human coordination mechanism rather than the natural unit of engineering work.

Why the sprint made sense

The sprint is a Scrum mechanism rather than an Agile principle. The Agile Manifesto asks for working software delivered frequently and at a sustainable pace, not for fixed iterations, which is precisely why the cadence can change without the principles changing.

The sprint is an elegant response to a human production system in which requirements need clarification, developers need time to implement, testers need working functionality, defects need correction, dependencies require synchronization, and stakeholders need predictable review points. A fixed cadence creates structure around limited capacity and therefore has real organizational value. The duration of the technical loop, however, was shaped partly by the amount of human effort required to complete it, so if that effort changes substantially, the optimal cadence of engineering may change with it.

Research before implementation

Hyper Velocity Engineering provides a useful signal of how AI-native engineering may evolve. HVE Core follows a Research, Plan, Implement, Review pattern in which the agent does not simply receive a request and begin producing code: it first investigates the relevant context, creates an evidence-based plan, implements against that plan, and then subjects the result to review.

The important lesson is not the name of the methodology but the sequencing. Agentic engineering does not need to abandon discipline in order to achieve speed; it can afford to apply disciplined steps more frequently because each step becomes less expensive. Research can happen repeatedly, plans can be regenerated when evidence changes, and review can become part of every significant technical cycle rather than a separate phase performed after large amounts of implementation have accumulated. This supports the idea of High-Frequency Rigor: disciplined engineering practices applied at a cadence that was previously uneconomic.

This point requires nuance, because current HVE Core documentation itself operates inside a broader lifecycle that can include sprint planning, delivery, review, and operations. Agentic engineering can therefore coexist with sprint-based organizational coordination, and there is no need to claim that sprints will disappear simply because technical execution becomes faster. The more interesting question is what happens inside the sprint. A two-week sprint may contain many Research, Plan, Implement, Review cycles, each producing validated technical progress, which changes the meaning of iteration even if the calendar remains unchanged.

Cadence decoupling

The agentic operating model therefore allows several clocks to coexist.

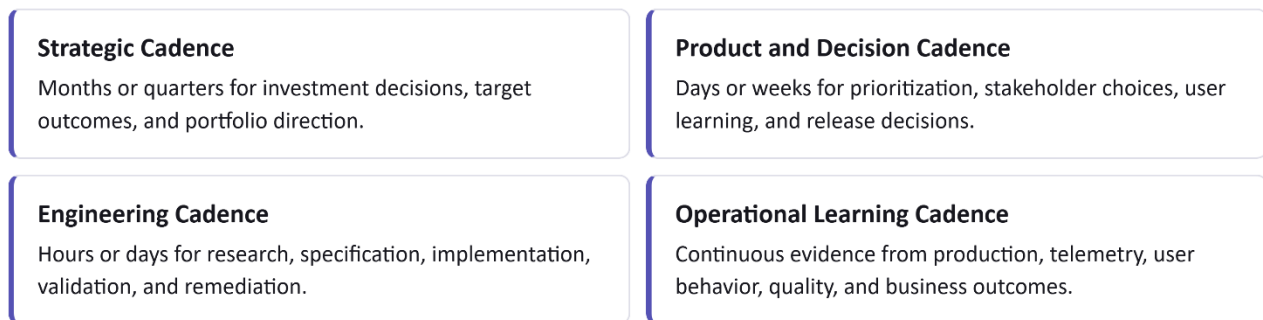


Figure 8. Decoupled cadences.

Individual agent actions may operate in minutes beneath those cadences, and the important implication is that the clocks no longer need to be artificially synchronized. Human organizations still need forums for prioritization, governance, communication, and alignment, but the technical system should not wait for a calendar boundary when there is no technical reason to wait.

*Agentic engineering may decouple engineering cadence
from management cadence.*

This means the agentic era may actually make engineering more iterative than traditional Agile. A conventional team may complete one significant build-test-learn loop within a sprint, while an agentic system may complete many such loops before the next stakeholder review point. The shift is therefore not from iterative to non-iterative development but from relatively coarse-grained iteration to much finer-grained, continuously validated iteration, and the engineering system becomes more fluid even if the organization's governance and product-management rhythms remain relatively stable.

Rethinking velocity

This change also challenges traditional measures of velocity. Story points are already an imperfect proxy for human capacity, and their meaning becomes even less stable when agentic capacity varies dramatically according to task type, context quality, tooling, and autonomy. Lines of code become effectively meaningless, while counting completed tasks can reward artificial fragmentation rather than business progress.

A more useful concept is Decision-to-Value Velocity, which measures how quickly the organization can move from uncertainty to evidence, from evidence to decision, from decision to implementation, and from implementation to validated outcome. This shifts the focus from how much work the delivery system appears to consume toward how effectively the organization converts knowledge into value. It needs no

new tooling to begin with: measure the elapsed time at each of those four transitions, and the measure also shows where the delay actually sits.

Hybrid Agile, reinterpreted

There may still be value in describing the future model as hybrid, but the term needs to be reinterpreted. Hybrid should not mean several months of Waterfall analysis followed by traditional Agile development, because that would simply reproduce the old methodological divide.

A more useful model is design-led and continuously adaptive. The program establishes structural foundations quickly, detailed engineering then moves through rapid agentic cycles, human product decisions operate at their own cadence, operational evidence challenges assumptions, and structural discoveries can escalate back to architecture when necessary. The system has enough stability to accelerate without pretending that every detail can or should be known in advance.

10. From Construction to Composition

Agentic engineering amplifies a trend already visible in low-code, cloud, and managed platforms: more engineering value is moving from constructing commodity capabilities toward selecting, combining, and governing existing ones.

Engineering has already moved up the stack

Modern enterprise teams rarely build everything themselves: identity services, databases, messaging, workflow engines, observability, API management, and infrastructure increasingly arrive as managed capabilities. Low-code and business application platforms push this abstraction further by allowing teams to assemble substantial portions of a solution through configuration and composition rather than traditional custom development.

Microsoft Power Platform provides a useful example because many elements of a business solution can coexist within one governed ecosystem. Dataverse provides managed data capabilities, Power Apps provides several application models, Power Automate provides workflow and orchestration, Power Pages provides external experiences, Copilot Studio provides agent capabilities, connectors expose services, APIs extend the platform, and pipelines automate important parts of application lifecycle management. This already changes the engineering problem: the team spends less time constructing commodity infrastructure and more time deciding which managed capability should own which responsibility.

Plans show the abstraction shift

Power Apps Plans makes the change in abstraction particularly visible because a maker can describe a business problem and receive a proposed solution containing requirements, data structures, and Power Platform components such as Dataverse, Power Apps, Power Automate, Power Pages, and Copilot Studio. The significance of this capability is not that complex enterprise planning can now be delegated to a product feature, since the Plans experience still depends on users describing the business problem clearly, reviewing the proposed plan, and refining it iteratively. The more important signal is that solution formulation, data modeling, and technical composition are moving closer to natural-language intent.

This strengthens rather than weakens the argument for an upstream Consulting Harness, because if downstream planning becomes cheaper, the quality of the problem statement feeding it becomes more consequential. Power Apps Plans can accelerate the formulation of a solution, but it does not independently determine whether the strategic objective is valid, whether the operating model should change, whether the underlying process deserves to exist, or whether the investment should be made at all.

APIs, connectors, and MCP widen what can be composed

REST APIs, connectors, and MCP expand the number of capabilities that agents can invoke without requiring a developer to implement every integration path from scratch. This matters particularly when the agent also

has access to architectural context, because the relevant question is no longer merely whether a function can be called.

The engineering system increasingly needs to determine whether it should be called, whether a deterministic flow would be more appropriate, whether an existing connector already satisfies the requirement, whether MCP is appropriate because the available tool surface is dynamic, which authentication model applies, which data-policy constraints govern the interaction, and which licensing and capacity consequences the choice carries. Composition therefore reduces some forms of construction while increasing the importance of architectural judgment.

Coding agents move into the platform

Power Platform skills for coding agents provide another glimpse of this direction. Packaged as plugins for GitHub Copilot and Claude Code, they give the agent the platform-specific knowledge and workflows to move from a natural-language description through planning, scaffolding, connector configuration, implementation, and deployment, whether the result is a canvas app, a model-driven app, a code app, a Power Pages site, or a cloud flow.

The significant change is not that developers disappear but that the agent can increasingly rely on the platform for identity, governance, telemetry, deployment, and service integration rather than constructing those concerns independently. The abstraction boundary moves upward, leaving the human engineer and architect to spend more attention on how the application should fit within the wider system.

Power Platform Pipelines provides a similar example on the lifecycle side, because it reduces the amount of specialist effort required to establish governed deployment paths. Environments, approvals, and deployment structures can be defined once and reused, allowing operational knowledge that previously lived in individual specialists to become part of the platform's reusable delivery capability. This is another form of encoded engineering knowledge, and it illustrates a broader pattern that extends beyond Power Platform: healthy engineering practices can increasingly be embedded into the environment rather than repeatedly reconstructed by each project.

From construction to composition

The engineering question therefore evolves from "How should we build this?" toward a more architectural question: "Which existing capabilities should we compose, under which constraints, to realize the intended outcome?" A mature team must determine which capability belongs in the platform, which belongs in custom code, which belongs in deterministic automation, which belongs in an agent, which should be exposed through an API, which should become a reusable skill, and which requires custom engineering because available components do not fit.

This is a higher-order problem, because selecting among existing capabilities requires a coherent view of the architecture rather than expertise in only one component. The faster construction becomes, the more

expensive poor composition decisions can become, since the engineering system will faithfully and rapidly materialize them.

Power Platform as evidence, not as boundary

The argument is intentionally broader than Microsoft because equivalent shifts are visible across cloud services, SaaS platforms, libraries, APIs, agent frameworks, managed databases, integration platforms, and developer tooling. Power Platform is useful because it provides a concentrated example of how managed capabilities, low-code composition, agents, reusable skills, APIs, MCP, governance, and lifecycle tooling can converge inside an enterprise ecosystem.

It therefore gives the white paper a practical anchor without defining the limits of the thesis. The broader point is that as construction becomes increasingly compositional, architectural coherence and consulting judgment become more important, not less.

Part V: The Agentic Operating Model

11. Consulting and Engineering as One Agentic System

The Agentic Operating Model connects business justification, whole-system design, specification, rapid engineering, independent verification, and operational learning into one traceable system. Its purpose is not to replace one methodology with another but to ensure that each level of work inherits the context it needs while remaining able to adapt when evidence changes.

Two harnesses, one context

The model contains two major harnesses that solve different problems. The Agentic Consulting Harness addresses upstream questions concerning why the organization should act, what outcome matters, which capability is missing, which process needs redesign, which alternatives exist, what target state is justified, and which trade-offs are acceptable. The Agentic Engineering Harness addresses downstream questions concerning how that approved direction should be represented technically, how intent should become specification, how implementation should be decomposed, which capabilities should be composed, what agents should build, and how the result should be independently verified.

Both are harnesses in the sense of Chapter 4, each carrying context, method, tools, and control; what differs is their content. The consulting harness draws its context from enterprise evidence and strategy, its method from consulting and architecture practice, and centers its control on human decision rights. The engineering harness draws its context from the approved design, its method from specification and engineering practice, and centers its control on independent verification and deployment policy.

These harnesses should not operate as disconnected systems. The engineering harness needs to inherit business rationale, architecture, decisions, constraints, assumptions, and unresolved risks from the consulting harness, while operational evidence needs to be capable of moving upward again when implementation reveals that an assumption about the design or business case was wrong.

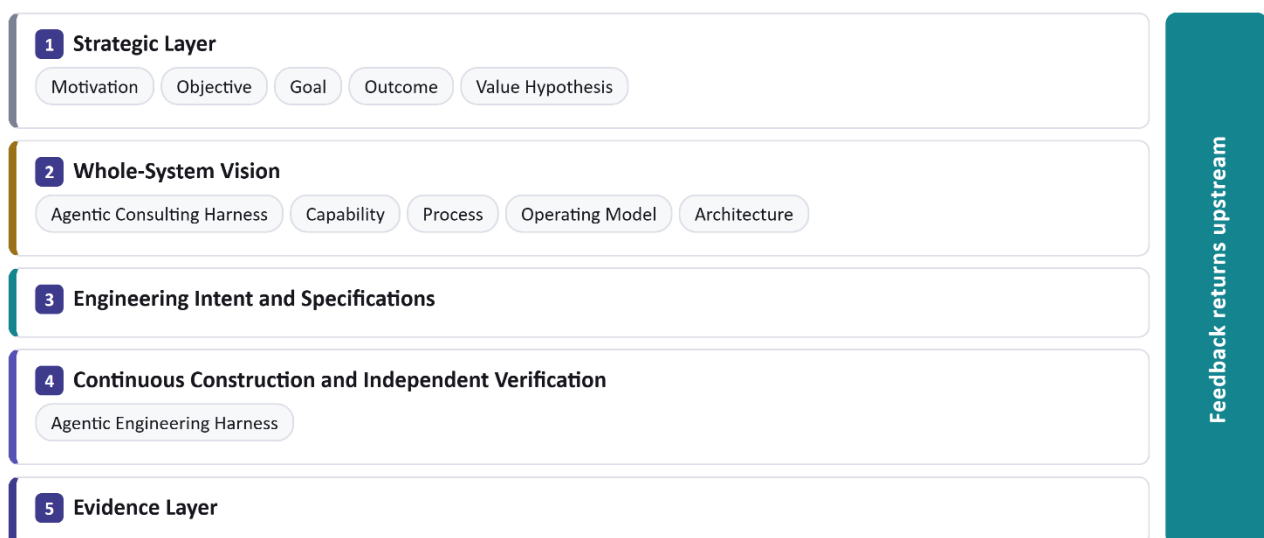


Figure 9. The Agentic Operating Model.

Transformation Mode

Transformation Mode applies when the system itself needs to be conceived or substantially reshaped. The logic begins with motivation and moves through objectives, measurable goals, expected outcomes, value hypotheses, capabilities, process and operating-model design, whole-system architecture, intent, specification, implementation, and evidence.

This should not be interpreted as a rigid stage-gate sequence. Different domains can mature at different rates, risky assumptions can be prototyped early, and selected capabilities can begin implementation before every aspect of the blueprint is complete. The important difference from fragmented delivery is that these experiments occur inside an evolving whole-system context, and their results are fed back into that context.

Continuous Improvement Mode

Continuous Improvement Mode operates differently because the context already exists. The system has known goals, its architecture is sufficiently established, and its main capability boundaries are understood, so a new intent can enter much lower in the model.

Intent can become specification, specification can become a plan, implementation can be created and verified, evidence can be collected, and the system can learn without reopening every strategic decision. Only when operational evidence challenges a structural assumption should feedback move higher into architecture, capability design, or strategy. This distinction is important because it prevents the Agentic Operating Model from becoming another bureaucracy: the amount of upstream work should be proportional to the structural significance of the decision being made.

Feedback must return to the right level

Agentic delivery needs more precision about feedback because not every problem is an implementation problem. Implementation feedback means that the solution does not implement the intended design correctly, so the appropriate response is to correct the implementation. Design feedback means that the specification or architecture is itself wrong, so the design needs to change. Strategic feedback means that the capability does not create the expected business outcome or that the underlying value hypothesis was invalid, in which case the organization may need to reconsider the capability, process, or objective itself.

Organizations frequently respond at the wrong level, so that a strategic failure becomes another feature request, an architectural problem becomes a defect backlog, or a flawed process becomes a usability issue. The operating model should use traceability and evidence to route learning toward the layer that can actually resolve it.

Classifying uncertainty

The model also needs a disciplined way to classify unknowns, because “we do not know” should not automatically result in another backlog item. A classification helps the organization decide whether the

correct action is to research, analyze, prototype, iterate, or simply accept uncertainty, rather than applying the same delivery mechanism to every unknown.

The unknown is	Action	Because
Known	Use evidence	Reliable evidence already exists
Researchable	Research	The answer exists but has not been obtained
Analytically derivable	Analyze	Modeling, comparison, or expert reasoning can resolve it
Experimentally knowable	Prototype	A simulation or prototype can test it economically
Operationally knowable	Iterate in real use	Only real-world use produces the evidence
Fundamentally uncertain	Manage risk	Reasonable analysis cannot remove it

Figure 10. Classifying uncertainty.

A known question already has reliable evidence and should therefore use that evidence. A researchable question has an answer that exists but has not yet been obtained, so investigation is appropriate. An analytically derivable question can be resolved through modeling, comparison, or expert reasoning, and an experimentally knowable question can be tested economically through simulation or prototype. An operationally knowable question requires real-world use, while a fundamentally uncertain question cannot be removed through reasonable analysis and therefore needs explicit risk management.

The six categories refine the two kinds of uncertainty in Chapter 2: known, researchable, and analytically derivable questions are avoidable uncertainty, while operationally knowable and fundamentally uncertain questions are genuine uncertainty. Experimentally knowable questions sit between them, and they are exactly where cheaper prototyping moves the line.

Front-loading without paralysis

This paper proposes an Agentic Front-Loading Canvas to assess whether the foundations are sufficient for acceleration without pretending that every category needs complete certainty. The purpose of the canvas is not to produce a perfect score before implementation begins but to make gaps explicit and to distinguish those that represent acceptable uncertainty from those that represent avoidable risk.

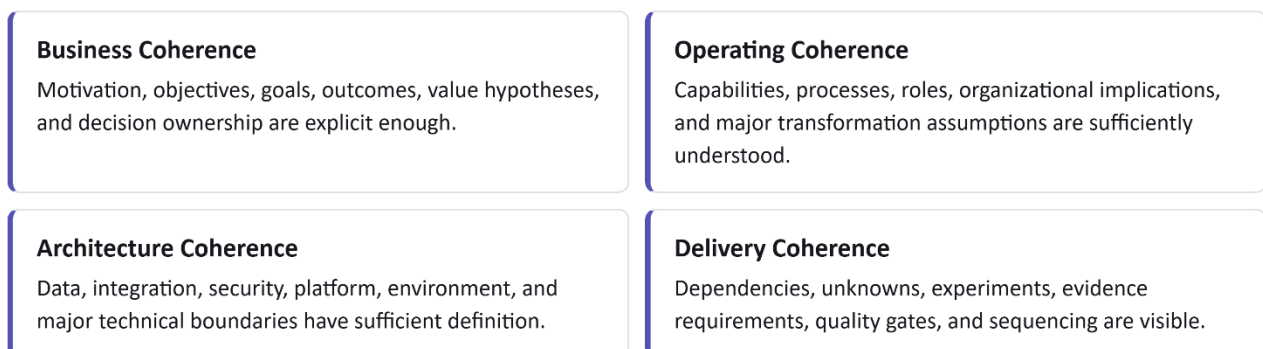


Figure 11. Agentic Front-Loading Canvas domains.

Independent verification

Increasing autonomy also creates a stronger verification requirement. An agent that generates a solution should not necessarily be the only mechanism that validates it, because using the same reasoning path for generation and certification can reproduce the same blind spots.

One agent can propose while another challenges the evidence, or implement while another reviews against the specification and architecture. A security evaluator can examine policy compliance, a traceability evaluator can verify alignment with intent, and a human can accept material residual risk where accountability requires human ownership. Independence, however, is not created by adding a second agent. A reviewer built on the same model, given the same context and prompted with the same assumptions, is a second opinion in name only; independence comes from different evidence, criteria the generator did not write, and, where the risk justifies it, a different model.

The principle is straightforward: the greater the autonomy of execution, the stronger the independence of verification. It becomes increasingly important as engineering speed rises, because errors that once propagated slowly can now be reproduced and deployed very quickly.

Beyond verification, agentic operating models also create an opportunity to move governance from periodic inspection toward continuously governed execution. Traditional governance often relies on boards and stage gates because manual governance is expensive, which means controls are frequently applied after significant design or implementation work has already occurred.

More of those expectations can increasingly be encoded into the environment. Architecture rules can become machine-readable checks, security requirements can become evaluators, traceability can be assessed continuously, deployment policy can be embedded in pipelines, and human governance can concentrate on exceptions and decisions with material consequences. The objective is not to eliminate governance but to reduce the amount of manual governance work required to maintain governed behavior.

12. The Return of Consulting

Agentic AI can automate meaningful portions of traditional delivery work, but this does not eliminate consulting. It makes the distinction between producing consulting artifacts and exercising consulting judgment increasingly visible.

From production scarcity to cognitive scarcity

Technology delivery has historically operated under conditions of production scarcity: there were never enough analysts, developers, testers, architects, integration and security specialists, project managers, or hours to perform all the work organizations wanted done. Large operating models grew up around that limitation. Work was divided into specialized roles, queues and backlogs managed demand, utilization governed professional-services economics, and delivery pyramids created leverage by combining many junior and mid-level practitioners with a smaller number of senior specialists.

Agentic systems weaken part of this relationship because they increase productive capacity, and as routine production becomes less scarce, a different constraint comes into view. This is cognitive scarcity: the shortage of people capable of consistently framing the right problem, challenging a false assumption, connecting business outcomes to architecture, recognizing hidden dependencies, understanding organizational consequences, evaluating difficult trade-offs, identifying what should not be automated, separating evidence from plausible narrative, and making a recommendation when information remains incomplete. AI can support these professionals by exposing more evidence, exploring more alternatives, preparing decision material, and maintaining context, but model capacity does not by itself create organizational judgment.

AI reduces the scarcity of production faster than it reduces the scarcity of judgment.

This shift puts pressure on technology-services models that connect revenue to the number of people required to perform work. McKinsey's 2025 analysis of technology services estimated that Agentic AI could compress significant portions of existing core service lines while creating new opportunities around agentic transformation and human-agent operating models. The precise outcome will depend on adoption, regulation, commercial structures, and client behavior, but the structural pressure is clear: more output will not necessarily require proportionally more people. I examine what that does to how consulting is priced, staffed, and funded, and why commercial models must reward leverage rather than effort, in *Why Power Platform Partners Must Evolve*; this chapter stays with what the same shift means for the quality of consulting itself.

Delivery knowledge versus consulting judgment

This separates two categories that professional-services organizations have historically sold together. Delivery knowledge includes knowing how to produce requirements, designs, configurations, code, tests, reports, documentation, and deployment artifacts, and much of this work can become increasingly automated or agent-assisted because it follows repeatable methods and can be evaluated against explicit criteria.

Consulting judgment, by contrast, includes knowing which problem deserves attention, which requirement should be challenged, which process should disappear, which architecture fits the organizational context, which trade-off is acceptable, which assumption lacks evidence, and which investment should not proceed. It is this second category that becomes relatively more valuable as production becomes cheaper.

SPOTLIGHT

Agentic AI commoditizes portions of delivery knowledge while increasing the premium on consulting judgment.

Weak consulting becomes visible

Methodology provides valuable structure, but it can also provide camouflage. A consultant can know how to facilitate a ceremony without understanding the client's business, and a Product Owner can manage backlog mechanics without connecting work to measurable outcomes. An architect can complete the expected template without understanding the system, and a team can create every required artifact while failing to make the right decision.

Agents can increasingly perform many visible methodological activities, including preparing workshop questions, summarizing meetings, creating backlog items, drafting specifications, generating plans, producing architecture alternatives, writing test cases, creating documentation, and preparing status reports. When those outputs become cheap, clients can ask a much more fundamental question: what judgment are we actually buying?

That question exposes weak consulting, but it also increases the leverage of strong consultants, because professionals who can challenge assumptions, understand the business, interpret ambiguity, navigate stakeholders, and make difficult recommendations can use the same agentic capability to operate at a much higher level. Their time moves away from copying workshop notes into documents, reconciling evidence, formatting deliverables, and tracing dependencies that a system can maintain, and toward asking better questions, framing decisions, evaluating trade-offs, communicating consequences, and taking professional responsibility for recommendations. The consultant does not disappear; the work becomes more recognizably consulting.

The same shift carries a cost for the profession. Junior consultants historically built pattern recognition by performing the very delivery work that agents may increasingly absorb, so the agentic era may solve part of

the delivery pyramid while destabilizing the learning pyramid. Challenging an agent's output requires the judgment that producing the work used to build, which means some production will need to remain deliberately human for people early in their careers, not because it is cheaper but because it is how judgment is formed. I set out the apprenticeship alternatives this requires in *Why Power Platform Partners Must Evolve*; the point here is that an organization optimizing only for machine leverage can become highly efficient at consuming expertise while failing to create the next generation of experts.

The role of the CoE

Centers of Excellence can become important institutional builders in this model because many of the capabilities required by an Agentic Operating Model should not be recreated independently by every project. A CoE can curate consulting methods, reference architectures, platform patterns, reusable skills, approved MCP integrations, agent tools, evaluation frameworks, security rules, HITL patterns, traceability conventions, testing standards, deployment patterns, observability, and governance.

The goal should not be to centralize every decision or create another approval bottleneck, but to encode high-quality behavior so that teams can reuse it, improve it, and apply it consistently. In this sense, the CoE becomes partly responsible for the continuous enhancement of the organization's consulting and engineering system itself.

The return of fundamentals

The deepest irony of Agentic AI is that increasingly sophisticated technology may increase the relative value of very old professional skills. Curiosity, problem framing, business understanding, architecture, critical thinking, communication, negotiation, commercial awareness, domain expertise, and judgment existed before Agile and before Waterfall, and they will remain relevant after today's AI architectures have changed completely.

The production mechanisms evolve. The professional fundamentals do not.

Conclusion - The Return of Competence

Enterprise technology delivery has spent decades managing an uncomfortable trade-off between understanding the whole and moving quickly enough for that understanding to remain useful. Traditional plan-driven programs often responded by extending analysis until the organization could no longer tolerate the delay, while Agile corrected the balance by bringing working software, feedback, and adaptation much closer to the beginning of the lifecycle.

That correction was necessary, but some organizations gradually allowed a useful principle to become a broader excuse. Unresolved analysis became uncertainty, deferred architecture became emergence, weak process understanding became backlog, and insufficient business justification became prioritization. Methodology could create movement even where consulting had failed to create direction.

Agentic AI changes the economics behind that compromise, but not because a language model can replace professional expertise or because uncertainty disappears. The change comes from the ability to engineer systems around models that combine enterprise context, reusable methods, specialized agents, tools, APIs, MCP, persistent decisions, independent verification, traceability, and Human-in-the-Loop decision experiences.

This changes upstream work because research, discovery, analysis, process design, architecture, and detailed design can potentially move faster while becoming more internally coherent. A mature organization should increasingly challenge whether the first credible blueprint, capability model, architecture baseline, SDD, functional design, TDDs, IDD, security model, data design, and major decision set truly require months to emerge under all circumstances.

Sometimes they will, because political conflict does not disappear, missing evidence does not disappear, regulatory ambiguity does not disappear, and human accountability does not disappear. What changes is the amount of mechanical work surrounding those problems and the amount of evidence a small number of skilled people can economically process before making a decision. The goal is therefore not to perform months of analysis in a few days merely to claim acceleration, but to establish a stronger foundation earlier, so that subsequent delivery begins from a coherent understanding of the business objectives, required capabilities, process direction, architecture, dependencies, constraints, and major decisions that define the system.

Downstream engineering changes at the same time. Agents can research before implementation, create plans, generate and configure software, compose managed capabilities, test, review, remediate, and deploy, while platforms increasingly provide identity, data, workflow, integration, agents, governance, and lifecycle services that teams once had to construct independently. The technical iteration becomes shorter, which creates the possibility of combining two ideas that enterprise delivery has too often treated as opposites: strong foundations and aggressive iteration.

A transformation can understand the whole without specifying every detail. It can establish motivations, objectives, goals, outcomes, capabilities, process direction, architecture, data, security, integration, platform boundaries, dependencies, and major decisions, and then allow implementation to iterate continuously inside that context. Whole-system thinking therefore does not eliminate Agile learning; it makes that learning more targeted by reserving iteration for areas where reality genuinely provides information that analysis cannot.

The same distinction positions Spec-Driven Development correctly. It can provide a strong mechanism for transforming intent into reliable implementation because it makes intent explicit and persistent. In a mature environment, intent can inherit an existing whole; in a transformation, the whole often needs to be established before meaningful intents can exist. The potential failure mode is not Spec-Driven Development itself but the possibility of adopting specifications with the same local optimization that previously affected some backlog-driven environments.

A perfect specification is not business justification, and a perfectly implemented specification can still represent the wrong investment. Consulting therefore sits above specification because motivation precedes intent, goals precede features, outcomes precede implementation, and capabilities precede technology composition. The organization must understand why something deserves to exist before engineering optimizes how quickly it can be created.

The construction model also deserves reconsideration: a two-week sprint may remain a useful human coordination mechanism, but it may not remain the natural technical clock. Strategic decisions can operate quarterly, product decisions weekly, engineering daily or hourly, agent execution in minutes, and operational learning continuously, and these cadences can separate without eliminating the organizational value of prioritization, governance, communication, and human alignment.

The future therefore does not require another universal methodology that replaces Agile with an equally rigid successor. It requires better judgment about which operating model fits which problem, how much of the whole can economically be understood before commitment, which uncertainties deserve experimentation, and where human accountability must remain central.

That returns the discussion to fundamentals. What problem are we solving, and why does it matter? What outcome justifies the investment, and what evidence do we possess? What can we know now, and what must we learn through experience? Which decisions require an understanding of the whole, and which details should remain adaptable? Where does human judgment add the most value, and where can agents safely execute? How should their output be independently verified, and how should evidence return to the level of decision-making that can actually respond to it? Those questions existed before Agile and before Waterfall, and Agentic AI does not make them obsolete. It changes the cost of answering them well.

That may ultimately be the most important implication for consulting. When producing a requirement becomes cheap, the value lies in knowing whether it is the right requirement, and when producing a specification becomes cheap, the value lies in knowing whether it is justified. When generating architecture alternatives becomes cheap, the value lies in choosing among them wisely; when producing software

becomes cheap, it lies in deciding what deserves to be built. And when methodology can be encoded into a harness, the value lies in the quality of the thinking encoded inside it.

The agentic era will not reward organizations merely because they can generate more software. It will reward those that can understand the whole, make better decisions, establish stronger foundations, and turn those decisions into validated outcomes faster than their competitors.

After Agile, the competitive advantage may no longer be the methodology. It may simply be competence.

Continue the Series

This white paper answered one question: what changes when organizations can reduce the cost of understanding and the cost of implementation at the same time. The Building the Center of Excellence for Power Platform series develops the governance, architecture, operating-model, organizational, and lifecycle foundations required to apply these ideas at enterprise scale.



Defining the Framework Structure for the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0GDDRCD2C>



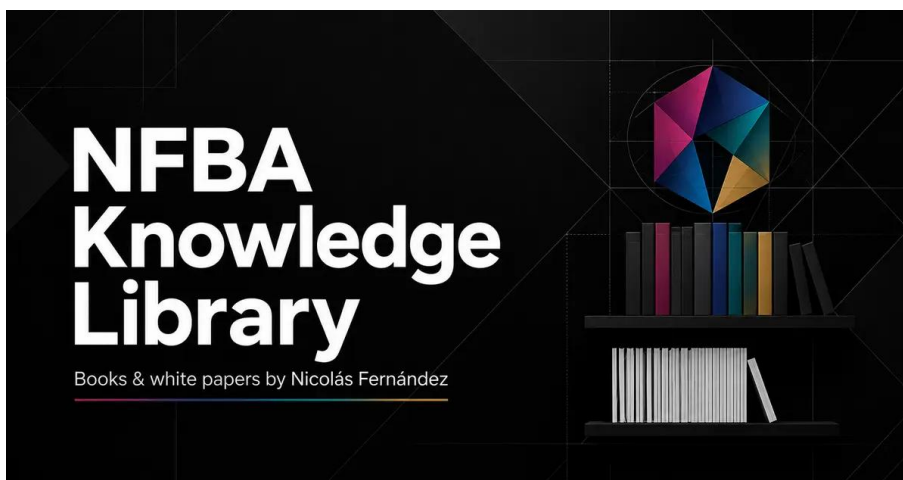
Copilot Studio and the Future of the Power Platform Center of Excellence

[View on Amazon ->](#)

<https://www.amazon.com/dp/B0H2SXQ3QQ>

Access to these and more material here:

<https://nfernandezba.github.io/Building-the-Center-of-Excellence-for-Power-Platform/>



Sources and References

Agile Manifesto. Principles behind the Agile Manifesto. Foundational principles covering frequent delivery, adaptation, collaboration, technical excellence, good design, simplicity, and continuous improvement.

Boston Consulting Group. Why Companies Get Agile Right and Wrong. May 8, 2024. Research across 127 companies examining differences between visible Agile adoption and sustained transformation outcomes.

DORA. State of AI-assisted Software Development 2025. Research examining AI adoption within software-delivery systems and the importance of the underlying organizational environment.

Fowler, Martin. The State of Agile Software in 2018. Discussion of faux-agile practices, technical excellence, organizational design, and divergence between Agile principles and superficial process adoption.

GitHub. Spec Kit: Agentic SDD. Current reference documentation describing Constitution, Specify, Clarify, Plan, Checklist, Tasks, Analyze, Implement, and Converge.

GitHub. Spec Kit: Spec of Specs. Current guidance for decomposing large features through an initial roadmap pass before independently specified subfeatures.

McKinsey & Company. Reimagining the Value Proposition of Tech Services for Agentic AI. December 2025. Analysis of potential changes to technology-services economics, human-agent delivery models, and the shift toward higher-value strategic activity.

Microsoft. power-platform-skills. GitHub repository of Power Platform plugins, skills, and agents for GitHub Copilot and Claude Code, covering canvas apps, model-driven apps, code apps, Power Pages, and Power Automate.

Microsoft for Developers. Spec-Driven Development: A Spec-First Approach to AI-Native Engineering. June 10, 2026. Discussion of structured specifications as shared context connecting intent, architecture, implementation, testing, and validation.

Microsoft HVE Core. Getting Started with HVE Core and current HVE project-lifecycle documentation. Documentation for Research, Plan, Implement, Review and its integration into a broader engineering lifecycle.

Microsoft Learn. Use plans to create AI-Powered business solutions with Copilot. Current Power Apps documentation for Plans, including generation of Dataverse, Power Apps, Power Automate, Power Pages, and Copilot Studio solution components.

Microsoft Learn. FAQ for plans in Power Apps. Current guidance on intended usage, iteration, limitations, and responsible operation of the Plans experience.

Microsoft Learn. Add tools and resources from a Model Context Protocol server to your agent. Current Copilot Studio documentation describing MCP tools and resources.

Microsoft Learn. Extend your agent with tools from a REST API. Current prerelease documentation describing REST API tools based on OpenAPI definitions and authentication.

Microsoft Learn. Quickstart: Build a code app with GitHub Copilot. June 2026. Documentation demonstrating Power Platform skills used by a coding agent to plan, create, connect, and deploy a Power Apps code application.

Microsoft Learn. Overview of pipelines in Power Platform. Current documentation for managed ALM, governed deployment, CI/CD, and platform pipeline extensibility.

Technology capabilities, terminology, preview status, product behavior, licensing, and availability can change. Product-specific references in this white paper were checked against official documentation available in September 2026 and should be verified again before implementation.